

Accessing Ipeadata data via API

Extracting the data

This tutorial is intended to help researchers retrieve data from Ipeadata by accessing its API. In essence, this repository contains three scripts: **Ipeadata_Query**¹, **Ipeadata_URL**² and **Ipeadata_API**³. The latter two functions form the core of the data retrieval process. Specifically, **Ipeadata_URL** generates the appropriate API request URL, while the **Ipeadata_API** collects the data. Finally, the **Ipeadata_Query** function combines these functions into a single interface, making data extraction considerably easier.

```
ipeadata_query(ipeadata_series_code, ipeadata_series_name, time_interval, source_github = TRUE)
```

Put into few words, the researcher only needs to provide a few arguments to the **Ipeadata_Query** function:

- `ipeadata_series_code`: code of the series to be extracted.
- `ipeadata_series_name`: name of the series to be extracted.
- `time_interval`: period⁴ (years) corresponding the data to be extracted.
- `source_github`: this optional argument allows the researcher to choose whether to source the function from this repo or from a local folder.

Let's consider an example. Suppose we want to extract the Brazilian Price Index and also its GDP for the period 2015-2025. While the former is a monthly series, the latter is only available at an annual frequency.

¹https://github.com/paulo-icaro/Ipeadata_API/blob/main/Ipeadata_Query.R

²https://github.com/paulo-icaro/Ipeadata_API/blob/main/Ipeadata_URL.R

³https://github.com/paulo-icaro/Ipeadata_API/blob/main/Ipeadata_API.R

⁴Naturally, Ipeadata provides datasets with different temporal frequencies. However, under the default extraction procedure, the data are filtered solely according to the years covered by the selected series.

```

# ===== #
# === Data Extraction === #
# ===== #

# --- Ipeadata Query Function --- #
source('https://raw.githubusercontent.com/paulo-icaro/Ipeadata_API/refs/heads/main/Ipeadata_Query.R')

# --- Previous Info --- #
series_code = c('PRECOS12_IPCA12', 'WEO_PIBWEOBRA')
series_name = c('br_price_index', 'br_gdp')
period = as.character(x = 2015:2025)

database = ipeadata_query(series_code, series_name, period)
print(head(database, 15), row.names = FALSE)

```

	data	br_price_index	br_gdp
2015-01-01T00:00:00-02:00		4110.20	1800.046
2015-02-01T00:00:00-02:00		4160.34	NA
2015-03-01T00:00:00-03:00		4215.26	NA
2015-04-01T00:00:00-03:00		4245.19	NA
2015-05-01T00:00:00-03:00		4276.60	NA
2015-06-01T00:00:00-03:00		4310.39	NA
2015-07-01T00:00:00-03:00		4337.11	NA
2015-08-01T00:00:00-03:00		4346.65	NA
2015-09-01T00:00:00-03:00		4370.12	NA
2015-10-01T00:00:00-03:00		4405.95	NA
2015-11-01T00:00:00-02:00		4450.45	NA
2015-12-01T00:00:00-02:00		4493.17	NA
2016-01-01T00:00:00-02:00		4550.23	1796.622
2016-02-01T00:00:00-02:00		4591.18	NA
2016-03-01T00:00:00-03:00		4610.92	NA

The fact that the two time-series have different frequencies imply an unbalanced dataset. This does not pose a major problem, provided some precautions are taken. In this example, the resulting dataset contains 131 monthly observations for the price index series and 10 annual observations, with each annual value assigned to the first month of the of its corresponding year. Under these circumstances, no information is lost, since all observations from both series are retained.

Conversely, if the GDP series were selected first, significant problems would arise. In that case, the resulting dataset would contain only 10 observations corresponding to the annual GDP values and the first month Price Index of each year. Consequently, most of the monthly info from Price Index series would be discarded.

Therefore, two approaches can be adopted to avoid this issue:

- When specifying the series to be retrieved, **always define the variables so that the highest-frequency series is listed first**.
- Retrieve series separately according to their temporal frequency. This approach requires calling the **Ipeadata_Query** function more than once.